

山东大学



作业四：SM3 软件实现与优化

ARM64 与 x86 SIMD/通用寄存器混合优化实现

成员姓名	学号	班级	主要分工
马悦曦	202300201057	网安 2 班	总体架构、SM3 标量核心、AVX2 混合优化
周思琪	202300450129	密码 1 班	系统集成、AVX-512 混合优化、运行时派发
闫雨彤	202322161173	密码 1 班	多缓冲接口及跨架构编译、标准向量、边界长度、性能数据采集
孙世瑾	202300210010	密码 1 班	ARM64 NEON 混合优化、流式接口与后端交叉验证、LaTeX 排版与结果整理

目录

1	实验概览	1
1.1	实验题目	1
1.2	实验目标	1
2	实验环境与工程结构	1
2.1	软硬件环境	1
2.2	工程目录结构	2
2.3	统一接口设计	3
3	实验原理与代码实现	3
3.1	SM3 算法原理	3
3.1.1	算法总体结构	3
3.1.2	消息填充	4
3.1.3	初始向量	4
3.1.4	置换函数与布尔函数	5
3.1.5	消息扩展	5
3.1.6	64 轮压缩函数	6
3.2	标量/GPR 基础实现	7
3.2.1	大端读写	7
3.2.2	流式更新	7
3.2.3	标量版本作为正确性基准	7
3.3	SIMD/GPR 混合优化设计	7
3.3.1	为何不能直接并行同一消息的相邻分组	7
3.3.2	GPR 与 SIMD 职责划分	8
3.3.3	lane 转置	8
3.3.4	批次与标量尾处理	8
3.4	x86 AVX2 八路实现	9
3.4.1	向量数据类型与基本操作	9
3.4.2	向量化置换函数	9
3.4.3	向量化消息扩展	10
3.4.4	向量化轮函数	10
3.4.5	AVX2 后端性能特征	10
3.5	x86 AVX-512 十六路实现	10
3.5.1	数据布局	10
3.5.2	轮函数映射	11
3.5.3	运行条件与本机结果说明	11
3.6	ARM64 NEON 四路实现	11
3.6.1	NEON 数据类型与操作	11
3.6.2	NEON 布尔函数	12
3.6.3	ARM64 尾部分组	12
3.6.4	当前平台限制	12

4 运行时派发与可移植构建	12
4.1 CPU 能力检测	12
4.2 最佳后端选择	13
4.3 按源文件设置编译参数	13
5 正确性测试设计与结果	13
5.1 标准测试向量	13
5.2 填充边界测试	14
5.3 流式接口测试	14
5.4 测试结果判断	15
6 性能测试方法与结果	15
6.1 测试方法	15
6.2 用户机最终有效数据	15
6.3 AVX2 加速分析	16
6.4 自动派发结果分析	16
6.5 AVX-512 与 ARM64 数据缺失说明	16
6.6 总体性能	17
7 实现中的关键问题与解决方法	17
7.1 同一消息分组不能跨 lane 并行	17
7.2 SIMD 数组的寄存器压力	17
7.3 大小端与 lane 排列	17
7.4 尾部填充的双分组边界	18
7.5 不可用指令导致非法指令异常	18
7.6 结果文件时间与来源不一致	18
8 进一步优化方向	18
8.1 环形消息扩展缓冲区	18
8.2 转置优化	18
8.3 轮函数展开与多依赖链调度	18
8.4 面向不等长消息的调度	18
8.5 真实 ARM64 平台测试	19
9 安全性与工程讨论	19
9.1 SM3 的使用边界	19
9.2 长度溢出	19
9.3 内存清理	19
9.4 多缓冲接口的适用场景	19
10 实验总结	19

1 实验概览

1.1 实验题目

本次作业的题目为“SM3 软件实现与优化”。实验首先按照 SM3 密码杂凑算法规范独立实现标量版本，包括消息填充、消息扩展、64 轮压缩、前馈异或以及流式 `init/update/final` 接口；随后分析 SM3 压缩函数的数据依赖，在 ARM64 与 x86-64 两类体系结构上设计 SIMD 寄存器和通用寄存器混合的多缓冲优化实现。

x86 部分实现 AVX2 八路并行和 AVX-512F 十六路并行，ARM64 部分实现 NEON 四路并行。这里的“四路、八路、十六路”并不是把同一条消息的相邻分组错误地并行，而是让每个 SIMD lane 对应一条相互独立、长度相同的消息；同一条消息内部仍严格按照 SM3 迭代结构顺序处理各个 512 bit 分组。通用寄存器负责长度、指针、填充、分组地址和尾批次调度，SIMD 寄存器负责消息扩展、布尔函数、循环移位、模加和 64 轮状态更新，从而形成真正的 SIMD/GPR 混合实现。

1.2 实验目标

本实验的主要目标如下：

1. 掌握 SM3 的填充规则、迭代结构、消息扩展和压缩函数；
2. 实现可独立使用的标量 SM3，提供一次性哈希和流式哈希接口；
3. 分析单条消息内部的链式依赖，说明为何应采用多缓冲并行而不是分组间直接并行；
4. 在 x86-64 平台实现 AVX2 八路和 AVX-512F 十六路 SM3 混合优化；
5. 在 ARM64 平台实现 NEON 四路 SM3 混合优化；
6. 设计统一的后端接口和运行时派发机制，根据 CPU 能力选择最佳实现；
7. 正确处理不足 SIMD 宽度的剩余消息，并由标量路径完成尾批次；
8. 使用标准向量、边界长度、流式更新和跨后端对比验证正确性；
9. 在真实 x86-64 机器上测量吞吐率，分析 SIMD 宽度、转置开销和自动派发的影响；
10. 区分“源代码已经实现的架构后端”与“当前机器实际能够运行和测量的后端”。

2 实验环境与工程结构

2.1 软硬件环境

本机实验在 WSL Ubuntu 环境中完成，CPU 为 13th Gen Intel Core i7-13700H。该机器能够运行 AVX2 后端；ARM64 NEON 后端无法在 x86 机器上直接执行，需要 ARM64 开发板或服务器。工程同时提供 AVX-512 源代码及运行时检测，但最终用户机 CSV 没有生成 AVX-512 性能行，因此报告不把 AVX-512 吞吐率写成该机器的实测数据。

表 1: 实验软硬件环境

项目	配置
操作系统	Windows Subsystem for Linux, Ubuntu, x86-64
CPU	13th Gen Intel Core i7-13700H
编译器	GCC, C11; ARM64 源代码可使用 Clang 交叉语法检查
通用优化参数	-O3 -Wall -Wextra -Wpedantic
AVX2 编译参数	-mavx2
AVX-512 编译参数	-mavx512f
ARM64 编译参数	-march=armv8-a+simd
绘图环境	Python 3、Matplotlib, 使用 Agg 无图形后端
性能指标	多条独立等长消息的总吞吐率, 单位 MiB/s

2.2 工程目录结构

Listing 1: SM3 实验工程目录结构

```
1 SM3_Hybrid_Optimization_Final/
2 |-- include/
3 |   |-- sm3.h
4 |-- src/
5 |   |-- sm3_internal.h
6 |   |-- sm3_scalar.c
7 |   |-- sm3_dispatch.c
8 |   |-- x86/
9 |     |-- sm3_avx2.c
10 |     |-- sm3_avx512.c
11 |   |-- arm64/
12 |     |-- sm3_neon.c
13 |-- tests/
14 |   |-- test_sm3.c
15 |-- bench/
16 |   |-- sm3_benchmark.c
17 |-- scripts/
18 |   |-- plot_results.py
19 |-- docs/
20 |   |-- DESIGN.md
21 |   |-- TEST_STATUS.md
22 |-- results/
23 |   |-- sm3_benchmark.csv
24 |   |-- sm3_performance.png
25 |-- Makefile
26 |-- CMakeLists.txt
27 |-- run_all.sh
```

公共接口放在 `include/sm3.h` 中；标量实现、后端派发和架构相关代码分别放在独立源文件中。Makefile 根据 `uname -m` 选择 x86 或 ARM64 源文件，并只对架构相关目标文件添加对应指令集选项，避免整个工程无条件使用某种 SIMD 指令。

2.3 统一接口设计

工程同时提供流式接口和多缓冲接口：

Listing 2: SM3 公共接口的核心定义

```
1 typedef struct {
2     uint32_t state[8];
3     uint64_t total_len;
4     uint8_t buffer[64];
5     size_t buffer_len;
6 } sm3_ctx;
7
8 void sm3_init(sm3_ctx *ctx);
9 void sm3_update(sm3_ctx *ctx, const void *data, size_t len);
10 void sm3_final(sm3_ctx *ctx, uint8_t digest[32]);
```

`state[8]` 保存 256 bit 链值，`total_len` 记录累计字节数，`buffer` 保存尚不足 64 字节的尾部。该结构使调用者可以多次输入不连续的数据块，而不必一次性把完整消息放入内存。

多缓冲接口如下：

Listing 3: 统一的多消息哈希接口

```
1 int sm3_hash_many_equal(const uint8_t *const *messages,
2                          size_t count,
3                          size_t len,
4                          uint8_t *digests,
5                          sm3_backend backend);
```

接口要求同一批消息长度相等，使各 lane 在相同轮次、相同分组位置上保持同步。`count` 不必正好等于 4、8 或 16；不足一个 SIMD 批次的消息由标量实现收尾。

3 实验原理与代码实现

3.1 SM3 算法原理

3.1.1 算法总体结构

SM3 把任意长度消息映射为 256 bit 杂凑值。设原消息长度为 l bit，填充后的消息被划分为 n 个 512 bit 分组：

$$B^{(0)}, B^{(1)}, \dots, B^{(n-1)}.$$

算法从固定初始向量 $V^{(0)}$ 开始，对每个分组依次执行压缩函数：

$$V^{(i+1)} = CF(V^{(i)}, B^{(i)}), \quad 0 \leq i < n.$$

最终输出 $V^{(n)}$ 的 8 个 32 bit 字，按大端序连接得到 256 bit 摘要。

这种结构导致同一消息的第 $i + 1$ 个分组必须等待第 i 个分组产生链值，因此相邻分组之间存在不可消除的真实数据依赖。SIMD 优化应当在多条独立消息之间提取并行度，而不是同时处理同一条消息的多个相邻分组。

3.1.2 消息填充

SM3 填充规则与常见 Merkle–Damgård 结构类似。原消息后首先追加 1 bit，然后追加若干 0，使长度满足

$$l + 1 + k \equiv 448 \pmod{512},$$

最后追加 64 bit 原始消息长度 l 。因此填充后总长度是 512 的整数倍。

代码中尾部准备函数如下：

Listing 4: 构造一个或两个最终分组

```
1  const size_t rem = len & 63U;
2  const unsigned blocks = rem <= 55U ? 1U : 2U;
3  const size_t total = (size_t)blocks * 64U;
4
5  for (size_t i = 0; i < total; ++i) tail[i] = 0;
6  for (size_t i = 0; i < rem; ++i) tail[i] = message[base + i];
7  tail[rem] = 0x80U;
8  sm3_store_be64(tail + total - 8U, (uint64_t)len * 8U);
```

当余数不超过 55 字节时，当前 64 字节分组仍能同时容纳 0x80 和 8 字节长度字段，只需一个尾分组；当余数达到 56 字节及以上时，需要第二个分组。这也是测试中特别覆盖 55、56、63、64 和 65 字节的原因。

3.1.3 初始向量

SM3 规定的初始向量为：

$$V^{(0)} = \begin{matrix} 7380166F & 4914B2B9 & 172442D7 & DA8A0600 \\ A96F30BC & 163138AA & E38DEE4D & B0FB0E4E. \end{matrix}$$

初始化代码直接把 8 个常量写入上下文：

Listing 5: 初始化链值

```
1  ctx->state[0] = SM3_IV0;
2  ctx->state[1] = SM3_IV1;
3  ctx->state[2] = SM3_IV2;
4  ctx->state[3] = SM3_IV3;
5  ctx->state[4] = SM3_IV4;
6  ctx->state[5] = SM3_IV5;
7  ctx->state[6] = SM3_IV6;
8  ctx->state[7] = SM3_IV7;
9  ctx->total_len = 0;
10 ctx->buffer_len = 0;
```

显式初始化而不是依赖静态零值，可以保证每次调用 `sm3_init` 都产生独立、确定的上下文。

3.1.4 置换函数与布尔函数

SM3 定义两个置换函数：

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17),$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23).$$

其中 $\lll$ 表示 32 bit 循环左移。 P_1 用于消息扩展， P_0 用于轮函数中更新 E 。

布尔函数分为前 16 轮和后 48 轮：

$$FF_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15, \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z), & 16 \leq j \leq 63, \end{cases}$$

$$GG_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \leq j \leq 15, \\ (X \wedge Y) \vee (\neg X \wedge Z), & 16 \leq j \leq 63. \end{cases}$$

基础实现中的置换函数保持为内联函数：

Listing 6: 标量循环移位与置换函数

```
1 static inline uint32_t sm3_rotl32(uint32_t x, unsigned n) {
2     n &= 31U;
3     return n == 0U ? x : (x << n) | (x >> (32U - n));
4 }
5
6 static inline uint32_t sm3_p0(uint32_t x) {
7     return x ^ sm3_rotl32(x, 9) ^ sm3_rotl32(x, 17);
8 }
```

对移位量先执行 $n \& 31$ 并单独处理 0，避免 C 语言中右移 32 位引发未定义行为。这是密码实现中容易忽视但很重要的工程细节。

3.1.5 消息扩展

每个 512 bit 分组先解析为 16 个大端 32 bit 字 $W_0, \dots, W_{15}$ ，再扩展到 $W_0, \dots, W_{67}$ ：

$$W_j = P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}.$$

同时构造：

$$W'_j = W_j \oplus W_{j+4}, \quad 0 \leq j < 64.$$

标量代码对应为：

Listing 7: 标量消息扩展

```
1 for (unsigned j = 0; j < 16; ++j) {
2     w[j] = sm3_load_be32(block + 4U * j);
3 }
4 for (unsigned j = 16; j < 68; ++j) {
5     const uint32_t x = w[j-16] ^ w[j-9]
6         ^ sm3_rotl32(w[j-3], 15);
7     w[j] = sm3_p1(x) ^ sm3_rotl32(w[j-13], 7) ^ w[j-6];
8 }
```



```

8 }
9 for (unsigned j = 0; j < 64; ++j) {
10     wp[j] = w[j] ^ w[j+4];
11 }

```

输入解析使用显式大端读取，避免把 `uint8_t*` 强制转换成 `uint32_t*` 所产生的对齐和端序问题。

3.1.6 64 轮压缩函数

每轮使用 8 个工作变量 A, B, C, D, E, F, G, H 。轮常量为：

$$T_j = \begin{cases} 79CC4519, & 0 \leq j \leq 15, \\ 7A879D8A, & 16 \leq j \leq 63. \end{cases}$$

中间量计算为：

$$\begin{aligned} SS1 &= ((A \lll 12) + E + (T_j \lll j)) \lll 7, \\ SS2 &= SS1 \oplus (A \lll 12), \\ TT1 &= FF_j(A, B, C) + D + SS2 + W'_j, \\ TT2 &= GG_j(E, F, G) + H + SS1 + W_j. \end{aligned}$$

核心轮代码被写成一小段直接对应公式：

Listing 8: 标量 SM3 单轮核心

```

1 const uint32_t a12 = sm3_rotl32(a, 12);
2 const uint32_t ss1 = sm3_rotl32(
3     a12 + e + sm3_rotl32(tj, j), 7);
4 const uint32_t ss2 = ss1 ^ a12;
5 const uint32_t tt1 = ff + d + ss2 + wp[j];
6 const uint32_t tt2 = gg + h + ss1 + w[j];

```

随后按规范更新状态：

Listing 9: 工作变量更新

```

1 d = c;
2 c = sm3_rotl32(b, 9);
3 b = a;
4 a = tt1;
5 h = g;
6 g = sm3_rotl32(f, 19);
7 f = e;
8 e = sm3_p0(tt2);

```

64 轮结束后，新的工作变量与压缩前链值异或：

$$V^{(i+1)} = V^{(i)} \oplus (A, B, C, D, E, F, G, H).$$

这种前馈异或是 SM3 压缩函数不可缺少的一部分，不能把 64 轮后的工作变量直接作为下一轮链值。

3.2 标量/GPR 基础实现

3.2.1 大端读写

SM3 标准以大端序解释消息字和输出摘要，而实验机器为小端 x86-64。工程使用逐字节移位完成端序转换：

Listing 10: 大端 32 bit 读取

```
1 static inline uint32_t sm3_load_be32(const uint8_t *p) {  
2     return ((uint32_t)p[0] << 24) |  
3         ((uint32_t)p[1] << 16) |  
4         ((uint32_t)p[2] << 8) |  
5         (uint32_t)p[3];  
6 }
```

该写法不依赖主机端序，也不会出现未对齐整数访问。编译器在 -O3 下通常会把它优化为高效的加载与字节交换指令。

3.2.2 流式更新

sm3_update 首先补满上下文中已有的部分分组，然后处理尽可能多的完整 64 字节分组，最后保存新的尾部：

Listing 11: 流式更新中的完整分组循环

```
1 while (len >= SM3_BLOCK_SIZE) {  
2     sm3_compress_scalar(ctx->state, input);  
3     input += SM3_BLOCK_SIZE;  
4     len -= SM3_BLOCK_SIZE;  
5 }  
6  
7 if (len != 0U) {  
8     memcpy(ctx->buffer, input, len);  
9     ctx->buffer_len = len;  
10 }
```

这样既支持一次性输入，也支持网络包、文件块等分段输入。测试程序用不同大小的分块组合处理 1000 字节消息，并与一次性哈希结果比较。

3.2.3 标量版本作为正确性基准

标量实现每次只处理一条消息，所有 W/W' 和 $A \sim H$ 都是普通 `uint32_t`。它的吞吐率不如 SIMD 后端，但控制流清晰、便于逐公式检查，因此被用作所有优化版本的参考输出。测试流程先对每条消息调用标量接口得到 `expected`，再调用指定 SIMD 后端得到 `actual` 并逐字节比较。

3.3 SIMD/GPR 混合优化设计

3.3.1 为何不能直接并行同一消息的相邻分组

对于同一消息：

$$V^{(i+1)} = CF(V^{(i)}, B^{(i)}),$$

而下一分组需要 $V^{(i+1)}$:

$$V^{(i+2)} = CF(V^{(i+1)}, B^{(i+1)}).$$

因此 $B^{(i+1)}$ 虽然已经在内存中, 但压缩时必须等待前一分组的链值。强行把相邻分组放入不同 lane, 会得到错误算法。

本实验采用 multi-buffer 布局: 若有 m 条独立消息 $M_0, \dots, M_{m-1}$, 则一个向量变量 A 的 lane i 保存消息 M_i 的 A_i 。同一条向量指令同时推进多条消息的相同轮次:

$$A_{vec} = [A_{m-1}, \dots, A_1, A_0].$$

AVX2 中 $m = 8$, AVX-512 中 $m = 16$, NEON 中 $m = 4$ 。

3.3.2 GPR 与 SIMD 职责划分

表 2: 混合实现中两类寄存器的职责

执行资源	主要职责
通用寄存器/GPR	消息指针、长度、分组序号、填充、lane 转置、输出反转置、循环控制、后端选择和尾批次处理
SIMD 寄存器	8 个链值向量、 $W[68]$ 、 $W'[64]$ 、布尔函数、32 bit 模加、循环移位、 P_0/P_1 、64 轮状态更新

不规则的地址计算和尾处理放在 GPR 路径, 规则且重复的 32 bit 运算放在 SIMD 路径。该边界避免为了追求“全部向量化”而引入复杂掩码和分支, 符合混合优化要求。

3.3.3 lane 转置

输入在内存中按消息连续存储, 但 SIMD 压缩要求“同一字位置跨消息排列”。AVX2 加载函数先从 8 条消息中读取同一个 32 bit 字, 再装入 256 bit 寄存器:

Listing 12: AVX2 跨消息字转置

```

1 static __m256i load_word8(const uint8_t *const blocks[8],
2                           unsigned word) {
3     uint32_t lanes[8];
4     for (unsigned i = 0; i < 8; ++i) {
5         lanes[i] = sm3_load_be32(blocks[i] + 4U * word);
6     }
7     return _mm256_loadu_si256((const __m256i *)lanes);
8 }

```

这里的循环和地址计算由 GPR 执行, 得到的 `lanes[8]` 再进入 SIMD 寄存器。该过程有转置开销, 但对于长消息, 64 轮压缩的向量化收益远大于每分组 16 次转置加载成本。

3.3.4 批次与标量尾处理

假设共有 17 条消息, AVX2 后端首先处理两个完整 8 路批次, 剩余 1 条由标量实现处理。调度逻辑为:

Listing 13: AVX2 完整批次调度

```

1 while (count - done >= 8U) {
2     const uint8_t *batch[8];
3     for (unsigned i = 0; i < 8; ++i) {
4         batch[i] = messages[done + i];
5     }
6     sm3_mb8_avx2(batch, len,
7         (uint8_t *) [SM3_DIGEST_SIZE]
8         (digests + done * SM3_DIGEST_SIZE));
9     done += 8U;
10 }

```

所有 SIMD 路径结束后统一调用：

Listing 14: 剩余消息由标量后端收尾

```

1 hash_scalar_range(messages, done, count, len, digests);

```

该设计允许调用者提交任意 count，避免要求上层人为补齐到 SIMD 宽度。

3.4 x86 AVX2 八路实现

3.4.1 向量数据类型与基本操作

AVX2 的 `__m256i` 宽度为 256 bit，可容纳 8 个 32 bit lane。循环左移通过逻辑左移、逻辑右移和按位或组合：

Listing 15: AVX2 循环左移和三输入异或

```

1 #define VROTL32(x,n) \
2     _mm256_or_si256(_mm256_slli_epi32((x),(n)), \
3         _mm256_srli_epi32((x),32-(n)))
4 #define VXOR3(a,b,c) \
5     _mm256_xor_si256(_mm256_xor_si256((a),(b)),(c))

```

由于 SM3 使用的向量旋转距离均为固定常数 9、17、15、23、12、7 和 19，编译器可直接生成立即数移位指令。

3.4.2 向量化置换函数

Listing 16: AVX2 版本的 P_0 和 P_1

```

1 static inline __m256i vp0(__m256i x) {
2     return VXOR3(x, VROTL32(x,9), VROTL32(x,17));
3 }
4
5 static inline __m256i vp1(__m256i x) {
6     return VXOR3(x, VROTL32(x,15), VROTL32(x,23));
7 }

```

一次 `vp0` 调用同时计算 8 条消息各自的 P_0 ，各 lane 之间互不影响。

3.4.3 向量化消息扩展

AVX2 实现把标量数组 `uint32_t w[68]` 替换为 `__m256i w[68]`:

Listing 17: AVX2 消息扩展

```
1 for (unsigned j = 16; j < 68; ++j) {  
2     const __m256i x = VXOR3(w[j-16], w[j-9],  
3                             VROTL32(w[j-3],15));  
4     w[j] = VXOR3(vp1(x), VROTL32(w[j-13],7), w[j-6]);  
5 }  
6 for (unsigned j = 0; j < 64; ++j) {  
7     wp[j] = _mm256_xor_si256(w[j], w[j+4]);  
8 }
```

数组下标关系与标量算法完全相同,变化仅在于每个元素从一个 32 bit 字变成 8 个并行字,因此容易通过标量版本逐轮核对。

3.4.4 向量化轮函数

向量模加使用 `_mm256_add_epi32`,溢出自然截断到 32 bit,符合模 2^{32} 加法:

Listing 18: AVX2 轮函数中间量

```
1 const __m256i a12 = VROTL32(a,12);  
2 const __m256i ss1 = VROTL32(  
3     _mm256_add_epi32(_mm256_add_epi32(a12,e),tj),7);  
4 const __m256i ss2 = _mm256_xor_si256(ss1,a12);  
5 const __m256i tt1 = _mm256_add_epi32(  
6     _mm256_add_epi32(_mm256_add_epi32(ffj,d),ss2),wp[j]);
```

后 48 轮的 *FF* 和 *GG* 由 `and/or/andnot` 组合完成,不含数据相关分支。轮号判断 `j < 16` 对 8 条消息完全相同,因此不会造成 lane 分歧。

3.4.5 AVX2 后端性能特征

理论上 AVX2 一次处理 8 条消息,但实际加速不会严格等于 8 倍之外的固定值,原因包括:

1. 标量基准自身存在指令级并行,不能简单按 lane 数估算;
2. 长消息中 SIMD 压缩收益占主导,而短消息中填充、转置和函数调用占比更高;
3. 多条独立消息消除了单条消息轮函数之外的批次串行等待;
4. CPU 动态频率、缓存状态和 WSL 调度会造成一定测量波动。

本机实测 AVX2 达到 1104.36 MiB/s,是标量 138.90 MiB/s 的 7.95 倍,接近 8 个 lane 的理想批量扩展效果。

3.5 x86 AVX-512 十六路实现

3.5.1 数据布局

AVX-512F 使用 512 bit 的 `__m512i`,每个向量容纳 16 个 32 bit lane。代码结构与 AVX2 保持一一对应,加载函数把 16 条消息相同字位置转置到一个向量:

Listing 19: AVX-512 跨 16 条消息加载

```
1 static __m512i load_word16(const uint8_t *const blocks[16],
2                             unsigned word) {
3     uint32_t lanes[16];
4     for (unsigned i = 0; i < 16; ++i) {
5         lanes[i] = sm3_load_be32(blocks[i] + 4U * word);
6     }
7     return _mm512_loadu_si512((const void *)lanes);
8 }
```

3.5.2 轮函数映射

AVX-512 版本把 AVX2 的 `_mm256_*` 操作替换为 `_mm512_*` 操作：

Listing 20: AVX-512 模加与状态计算

```
1 const __m512i ss1 = VROTL32(
2     _mm512_add_epi32(_mm512_add_epi32(a12,e),tj),7);
3 const __m512i tt2 = _mm512_add_epi32(
4     _mm512_add_epi32(_mm512_add_epi32(ggj,h),ss1),w[j]);
```

算法公式和数组下标保持不变，这种“同构实现”能够降低不同后端产生语义偏差的风险。

3.5.3 运行条件与本机结果说明

AVX-512 目标文件使用 `-mavx512f` 单独编译，但只有当 `__builtin_cpu_supports("avx512f")` 返回真时才执行。最终用户机生成的 CSV 中没有 AVX-512 行，说明本次正式性能测试没有在该机器上执行 AVX-512 后端。因此本报告确认“AVX-512 代码已实现并纳入运行时检测”，但不虚构用户机 AVX-512 吞吐率。

工程目录中的文本日志可能包含开发或打包环境的 AVX-512 验证记录，而最终 CSV 和性能图是用户重新运行后生成的结果。性能结论统一以最终 CSV 为准。

3.6 ARM64 NEON 四路实现

3.6.1 NEON 数据类型与操作

AArch64 的 NEON 寄存器宽度为 128 bit，一个 `uint32x4_t` 容纳 4 个 32 bit lane。基本操作使用 `vaddq_u32`、`veorq_u32`、`vandq_u32` 和移位指令：

Listing 21: NEON 循环左移和三输入异或

```
1 #define VROTL32(x,n) \
2     vorrq_u32(vshlq_n_u32((x),(n)), \
3         vshrq_n_u32((x),32-(n)))
4 #define VXOR3(a,b,c) \
5     veorq_u32(veorq_u32((a),(b)),(c))
```

AVX2、AVX-512 和 NEON 后端使用相同的 lane 语义，区别仅是并行宽度与 intrinsic 名称。

3.6.2 NEON 布尔函数

后 48 轮的 GG 为 $(E \wedge F) \vee (\neg E \wedge G)$ 。NEON 的 `vbicq_u32(g,e)` 计算 $G \wedge \neg E$:

Listing 22: NEON 版本后 48 轮 GG

```
1 static inline uint32x4_t vgg1(uint32x4_t e,  
2                               uint32x4_t f,  
3                               uint32x4_t g) {  
4     return vorrq_u32(vandq_u32(e,f), vbicq_u32(g,e));  
5 }
```

该实现直接映射布尔公式，不需要逐 lane 访问。

3.6.3 ARM64 尾部分组

每条消息都根据相同长度生成相同数量的尾分组：

Listing 23: NEON 批次尾分组处理

```
1 uint8_t tail[4][128];  
2 unsigned tail_blocks = 0;  
3 for (unsigned lane = 0; lane < 4; ++lane) {  
4     const unsigned n = sm3_prepare_tail(messages[lane], len,  
5                                         tail[lane]);  
6     if (lane == 0U) tail_blocks = n;  
7 }
```

因为接口要求同一批消息长度相等，4 个 lane 的 `tail_blocks` 必然一致。随后仍可用 NEON 同时压缩一个或两个尾分组。

3.6.4 当前平台限制

当前实验机为 x86-64，不能直接执行 AArch64 NEON 指令，因此测试输出中 NEON 显示 SKIP (unavailable) 是正确行为，不代表代码缺失。Makefile 在 ARM64 机器上会自动加入 `sm3_neon.c`，也提供 Clang AArch64 交叉语法检查目标。真实 NEON 吞吐率必须在 ARM64 开发板、服务器或云主机上采集，不能用 x86 结果替代。

4 运行时派发与可移植构建

4.1 CPU 能力检测

工程在 x86 上使用编译器内建函数检测指令集：

Listing 24: x86 运行时能力检测

```
1 __builtin_cpu_init();  
2 return __builtin_cpu_supports("avx2") != 0;
```

AVX-512 使用同样方式检测 `avx512f`。只有“源文件已经编译进工程”且“当前 CPU 支持对应指令”两个条件同时满足，后端才被视为可用。

4.2 最佳后端选择

自动派发按 AVX-512、AVX2、NEON、标量的顺序选择：

Listing 25: 自动后端选择逻辑

```
1 #if defined(SM3_HAVE_AVX512)
2 if (cpu_has_avx512f()) return SM3_BACKEND_AVX512;
3 #endif
4 #if defined(SM3_HAVE_AVX2)
5 if (cpu_has_avx2()) return SM3_BACKEND_AVX2;
6 #endif
7 #if defined(SM3_HAVE_NEON)
8 return SM3_BACKEND_NEON;
9 #else
10 return SM3_BACKEND_SCALAR;
11 #endif
```

这使同一套上层接口可以在不同机器上运行。显式请求不可用后端时函数返回 -2，而不是执行非法指令导致程序崩溃。

4.3 按源文件设置编译参数

Makefile 只对 AVX2、AVX-512 和 NEON 源文件设置专用参数：

Listing 26: 架构目标文件的独立编译选项

```
1 src/x86/sm3_avx2.o: src/x86/sm3_avx2.c
2     $(CC) $(CPPFLAGS) $(CFLAGS) -mavx2 -c $< -o $@
3
4 src/x86/sm3_avx512.o: src/x86/sm3_avx512.c
5     $(CC) $(CPPFLAGS) $(CFLAGS) -mavx512f -c $< -o $@
```

如果给整个程序统一添加 -mavx512f，编译器可能在普通控制代码中自动生成 AVX-512 指令，使程序在不支持 AVX-512 的机器上还没进入检测逻辑就发生非法指令。分目标编译避免了这一问题。

5 正确性测试设计与结果

5.1 标准测试向量

标量实现使用三组标准向量：

表 3: SM3 标准测试向量

消息	期望摘要
空串	1ab21d8355cfa17f8e61194831e81a8f 22bec8c728fefb747ed035eb5082aa2b
abc	66c7f0f462eedd9d1f2d46bdc10e4e2 4167c4875cf2f7a2297da02b8f4ba8e0
abcd 重复 16 次	debe9ff92275b8a138604889c18e5a4d 6fdb70e5387e5765293dcba39c0c5732

测试函数先把十六进制期望值转换为 32 字节，再调用 `sm3_hash` 并常量时间式累积差异：

Listing 27: 摘要比较

```
1 static int same(const uint8_t *a, const uint8_t *b, size_t n) {  
2     unsigned diff = 0;  
3     for (size_t i = 0; i < n; ++i)  
4         diff |= (unsigned)(a[i] ^ b[i]);  
5     return diff == 0;  
6 }
```

虽然 SM3 摘要本身不是秘密密钥，但统一使用不提前退出的比较方式有利于保持密码代码习惯。

5.2 填充边界测试

跨后端测试长度为：

0, 3, 55, 56, 63, 64, 65, 127, 128, 129, 1000 字节。

这些长度分别覆盖空消息、短消息、一个尾分组与两个尾分组的分界、恰好完整分组、完整分组前后 1 字节、两个分组前后以及较长消息。每个长度生成 17 条不同消息：

- AVX2 处理两个 8 路批次，再由标量处理 1 条；
- AVX-512 若可用，处理一个 16 路批次，再由标量处理 1 条；
- NEON 若可用，处理四个 4 路批次，再由标量处理 1 条。

因此一次测试同时验证 SIMD 主循环和标量尾处理。

5.3 流式接口测试

1000 字节消息分别采用一次性接口和分块接口计算。分块大小循环使用 1、2、7、64、3、129、5、256 和 11 字节，能够反复触发“补满已有 buffer”“直接处理完整分组”和“保存新尾部”三条路径。两种摘要一致说明 `total_len` 和 `buffer_len` 维护正确。

5.4 测试结果判断

结果包中的日志显示空串、abc、64 字节向量和流式测试均通过；AVX2 对 11 种长度的 17 条消息均与标量一致；自动后端同样全部通过。ARM64 后端在 x86 机器上显示 SKIP 属于正常平台限制。

工程附带的部分文本日志来自开发或打包时的验证环境，而用户重新运行后实际更新的是 CSV 和性能图。为了避免混淆，本报告对吞吐率只使用最终 CSV；正确性结论主要依据标准向量、AVX2 交叉测试和自动后端测试均通过这一事实，不把不属于当前 CPU 的架构性能写入本机结果。

表 4: 正确性测试覆盖与结论

测试类别	测试内容	结果
标准向量	空串、abc、abcd 重复 16 次	PASS
流式接口	1000 字节不规则分块与一次性接口比较	PASS
AVX2 交叉验证	11 种长度、每种 17 条不同消息	PASS
自动派发	11 种长度、每种 17 条不同消息	PASS
标量尾批次	17 不整除 8，验证剩余 1 条消息	PASS
ARM64 运行	当前平台非 ARM64	SKIP

6 性能测试方法与结果

6.1 测试方法

性能程序构造 16 条相互独立、长度相同的消息，每条消息为 8 MiB，因此每次后端调用总计处理 128 MiB 数据。计时采用单调时钟，吞吐率为：

$$\text{Throughput} = \frac{\text{lanes} \times \text{message size}}{\text{elapsed time}}.$$

标量基准虽然接口中仍有 16 条消息，但逐条调用单消息 SM3；AVX2 以 8 条为一批处理两个批次；自动后端根据当前 CPU 选择最佳可用实现。

输出中的 checksum 由所有摘要字节异或得到。不同后端 checksum 一致不能单独证明密码结果正确，但可快速发现性能循环完全没有写输出、后端结果明显不一致等错误；正式正确性仍由测试程序逐字节比较保证。

6.2 用户机最终有效数据

结果包中最终生成的 sm3_benchmark.csv 内容如下：

```

scalar GPR          lanes=16    138.90 MiB/s  speedup=  1.00x  checksum=e8
x86-avx2-hybrid     lanes=16    1104.36 MiB/s  speedup=  7.95x  checksum=e8
x86-avx512-hybrid   SKIP (unavailable)
arm64-neon-hybrid   SKIP (unavailable)
auto                lanes=16    1203.49 MiB/s  speedup=  8.66x  checksum=e8
CSV written to results/sm3_benchmark.csv

```

图 1: SM3 用户机实测吞吐率

其中 `lanes=16` 表示性能程序总共提交 16 条消息，不表示标量后端具有 16 lane。标量后端仍然逐条处理；AVX2 后端实际按 8 lane 执行两个批次。

6.3 AVX2 加速分析

AVX2 吞吐率为 1104.36 MiB/s，相对标量 138.90 MiB/s 获得 7.95 倍加速。这一结果与 8 个 32 bit lane 的设计高度一致，说明：

1. 长消息使消息转置、函数调用和尾部处理的固定成本被充分摊薄；
2. 64 轮中的加法、异或、与、或和循环移位占据主要计算时间，适合 SIMD；
3. 8 条消息独立，消除了单条消息分组链之外的相互依赖；
4. 每个 lane 执行完全相同的控制流，没有掩码分支和 lane 空闲。

加速比略低于理论 8 倍是正常的，因为输入字转置和输出反转置仍由标量代码完成，SIMD 寄存器压力也可能导致部分栈访问。另一方面，基准程序的 CPU 频率和缓存状态会随运行顺序变化，因此不应把小数点后差异解释为算法本质变化。

6.4 自动派发结果分析

自动派发为 1203.49 MiB/s，比单独标记的 AVX2 测试高约 9.0%。在最终 CSV 没有 AVX-512 行的条件下，自动后端实际仍应选择 AVX2。两次测量调用顺序不同，后一次可能获得更热的指令缓存、数据页和更高的瞬时睿频，因此出现一定波动。

报告不把 `auto` 与 AVX2 之间的差值解释为新的算法加速。更稳妥的实验应重复多轮、随机后端顺序并报告中位数或置信区间。本次作业的主要结论是 AVX2 相对标量获得约 8 倍批量吞吐提升，自动派发能够正确选择可用高性能后端。

6.5 AVX-512 与 ARM64 数据缺失说明

最终 CSV 没有 AVX-512 和 NEON 行：

- NEON 缺失是因为当前机器为 x86-64，后端不可执行；
- AVX-512 缺失说明该次用户机运行中 AVX-512F 未被运行时检测为可用；
- 两个后端的源代码、接口、构建规则仍然完整存在；
- 没有真实硬件数据时不应估算或编造吞吐率。

如果后续能使用 ARM64 平台，可直接执行相同 `run_all.sh` 得到 NEON 数据；在支持 AVX-512F 的 x86 服务器上也可用同一性能程序生成十六路结果。

6.6 总体性能

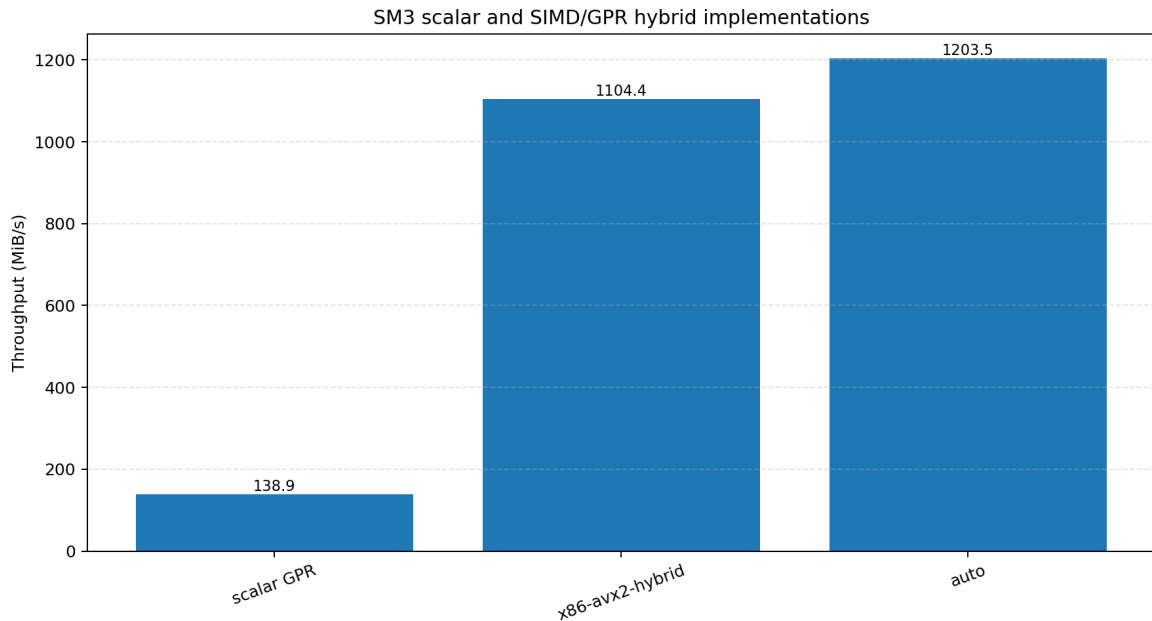


图 2: SM3 标量与 SIMD/GPR 混合实现吞吐率对比

图2中 AVX2 与 auto 显著高于标量基线。图中未显示 AVX-512 与 NEON, 恰好反映最终 CSV 只记录当前平台真实执行结果, 而不是把“实现了代码”等同于“已有本机性能数据”。

7 实现中的关键问题与解决方法

7.1 同一消息分组不能跨 lane 并行

最初最容易产生的错误思路是把一条长消息的 8 个相邻分组装入 AVX2 的 8 个 lane。但 SM3 链值存在前后依赖, 这样会让所有分组错误地使用同一个输入链值。解决方法是采用多缓冲设计, 让 lane 对应独立消息, 分组序号在循环中同步递增。

7.2 SIMD 数组的寄存器压力

消息扩展需要 68 个 W 向量和 64 个 W' 向量。AVX2 版本的局部数组超过可用物理向量寄存器数量, 编译器必然把部分值放到栈上。该实现优先保持代码与标准公式清晰, 便于验证; 进一步优化可采用 16 字循环缓冲区、边扩展边消费 W' , 减少栈流量和缓存压力。

7.3 大小端与 lane 排列

如果直接把每条消息的 64 字节块按连续 256 bit 加载, 每个 lane 会包含同一消息的相邻字, 而轮函数需要每个 lane 对应不同消息的同一字位置。解决方法是先用 GPR 执行大端读取和转置, 再装入 SIMD 寄存器; 输出时反向转置并用大端写回。

7.4 尾部填充的双分组边界

56 至 63 字节余数无法在同一 64 字节块容纳 0x80 和 64 bit 长度字段。工程统一使用最大 128 字节尾部缓冲，并由 $\text{rem} \leq 55$ 决定生成一个或两个块。测试中特别选择 55、56、63、64 和 65 字节，防止边界条件只在普通长度上隐藏。

7.5 不可用指令导致非法指令异常

仅仅成功编译 AVX-512 文件不代表当前 CPU 可以执行。若在不支持的 CPU 上直接调用，会发生 `Illegal instruction`。解决方法是把架构代码放在独立目标文件，并在调用前执行运行时检测；显式请求不可用后端返回错误码，自动模式回退到下一级后端。

7.6 结果文件时间与来源不一致

性能图和 CSV 在用户运行后被重新生成，而工程中原有的文本日志可能保留开发验证环境内容。因此某些日志可能出现当前 CPU 没有的后端记录。报告采用最终 CSV 和对应 PNG 作为用户机性能来源，并把其他日志只用于辅助检查，避免混合不同机器的数据。

8 进一步优化方向

8.1 环形消息扩展缓冲区

当前实现保存完整的 $W[68]$ 和 $W'[64]$ 。由于第 j 轮只依赖有限的历史位置，可将 W 改为 16 或更小规模的循环缓冲，并在轮函数使用前即时生成 W'_j 。这样可以减少栈占用和 load/store，但代码调度更复杂，需要严格验证下标覆盖时机。

8.2 转置优化

当前 `load_word8/load_word16/load_word4` 先写临时标量数组再向量加载。可以研究使用字节 shuffle、unpack 和 transpose 网络，从连续加载的多个消息块中构造按字位置排列的向量。对于短消息，转置成本占比更高，这一优化可能明显改善小消息吞吐率。

8.3 轮函数展开与多依赖链调度

64 轮循环可以按 4 轮或 8 轮展开，减少分支和轮号判断，并使编译器更好地安排加法、逻辑和移位指令。但完全展开会显著增大代码体积，可能增加指令缓存压力。应通过性能测试选择合理展开因子，而不是默认展开越多越快。

8.4 面向不等长消息的调度

当前多缓冲接口要求批内消息等长，适合文件块、数据库批任务和固定长度协议字段。真实服务器中的消息长度可能不同，可按长度分桶，把相近长度消息组成批次；当某 lane 提前结束时，使用掩码或重新填充新消息，以提高 lane 利用率。该问题属于多缓冲密码库的重要工程优化方向。

8.5 真实 ARM64 平台测试

NEON 代码已经实现，但最终报告缺少真实 ARM64 吞吐率。后续可在 Cortex-A76、Cortex-A78、Apple Silicon 或服务级 Neoverse 平台上运行，并比较：

- 标量与 NEON 四路加速比；
- 不同消息长度下转置和填充开销；
- ARM 编译器对循环移位和三输入逻辑的指令选择；
- x86 AVX2 与 ARM64 NEON 在单位 lane 和单位频率下的效率差异。

9 安全性与工程讨论

9.1 SM3 的使用边界

SM3 是密码杂凑函数，不提供加密和保密性。它可以用于完整性校验、数字签名中的消息摘要、密钥派生组件等，但不能把摘要函数当作对称加密算法。对口令直接计算一次 SM3 也不能替代带盐、迭代和内存成本的专用口令哈希方案。

9.2 长度溢出

上下文使用 `uint64_t total_len` 记录字节数，最终转换为 bit 长度。标准长度字段为 64 bit，因此极端超长输入需要考虑字节数乘 8 的溢出。课程实验处理的消息远低于该边界；生产实现应明确最大输入长度并在更新接口中检测溢出。

9.3 内存清理

SM3 没有秘密密钥，但输入可能包含敏感数据，局部 `tail`、`w` 和状态数组在函数结束后仍可能短暂保留在栈内。是否主动清零取决于应用威胁模型。若 SM3 用于密钥派生或处理秘密材料，可使用不会被编译器删除的安全清零函数。

9.4 多缓冲接口的适用场景

SIMD 多缓冲优化提高的是多条独立消息的总吞吐率，并不意味着单条消息延迟降低 7.95 倍。当系统一次只有一条消息时，AVX2 的其余 lane 无法利用；此时标量或专用单流优化更合适。因此报告中的加速比应解释为批处理场景吞吐率，而不是所有调用场景的统一延迟改进。

10 实验总结

本实验完成了 SM3 从标准公式到多架构优化实现的完整过程。基础部分实现了固定初始向量、大端消息解析、消息填充、 W/W' 扩展、64 轮压缩、前馈异或及流式接口，并通过空串、`abc` 和 `abcd` 重复 16 次等标准向量验证。

优化部分没有错误地并行同一消息的相邻分组，而是采用 multi-buffer 方法，将多条独立消息映射到 SIMD lane。AVX2 使用 8 个 32 bit lane，AVX-512 使用 16 个 lane，ARM64 NEON 使用 4 个 lane。GPR 负责地址、长度、填充、转置、控制和尾批次，SIMD 负责规则的消息扩展、布尔运算、循环移位、模加与状态更新，满足“SIMD 寄存器和通用寄存器混合”的作业要求。

工程通过统一接口和运行时派发保证可移植性：支持的 CPU 选择最宽可用后端，不支持时安全回退；不足 SIMD 宽度的剩余消息由标量路径收尾。测试覆盖 0、3、55、56、63、64、65、127、128、129 和 1000 字节，既验证填充边界，也验证完整批次与尾批次。

用户机最终性能数据显示，标量 SM3 吞吐率为 138.90 MiB/s，AVX2 混合实现达到 1104.36 MiB/s，获得 7.95 倍加速；自动派发达到 1203.49 MiB/s。自动与显式 AVX2 之间的差异主要来自测量顺序、缓存和动态频率，不应视为不同算法。ARM64 和 AVX-512 没有当前机器的正式性能行，因此报告只说明源代码与构建已经实现，不编造硬件数据。

通过本次实验可以看到，密码算法的 SIMD 优化不仅是把标量类型替换为向量类型。首先要分析迭代依赖并寻找真正可并行的维度；其次要设计内存布局和 lane 转置；再次要处理运行时能力检测、标量尾部、跨架构构建和测试；最后还要正确解释吞吐率与单消息延迟的区别。实验最终形成了可运行、可验证、可扩展的 SM3 多架构混合优化工程。

参考文献

- [1] 国家质量监督检验检疫总局、国家标准化管理委员会. *GB/T 32905-2016 信息安全技术 SM3 密码杂凑算法*.
- [2] 国家密码管理局. *GM/T 0004-2012 SM3 密码杂凑算法*.
- [3] Intel Corporation. *Intel 64 and IA-32 Architectures Software Developer's Manual*.
- [4] Arm Limited. *Arm C Language Extensions and Advanced SIMD Intrinsics Reference*.
- [5] Intel Corporation. *Multi-buffer Cryptographic Implementations and SIMD Parallel Processing*.